Recognising handwritten digits

🎯 Today's goal

By the end of this hour you will:

- see how a picture is stored as a grid of numbers
- show images with `plt.imshow`
- turn each 8×8 picture into 64 features and scale them
- train a neural network (`MLPClassifier`) that reads handwritten digits
- check it with accuracy and a confusion matrix, look at its mistakes and its confidence

🤔 Why this matters

Reading handwriting was one of the first big jobs for neural networks: in the 1990s, banks and post offices used them to read cheques and postcodes. Today the same idea, scaled up, lets your phone recognise faces and lets cars spot road signs.

Yesterday you built a network by hand for 4 examples. Today you'll use a ready-made one from scikit-learn on almost 1800 small pictures. Under the hood it does exactly what you wrote yesterday: forward pass, loss, backpropagation, gradient descent.

🖼️ A picture is just numbers

scikit-learn comes with a small dataset of handwritten digits (0 to 9), collected from real people's handwriting in the 1990s. It is built into the library, so nothing is downloaded. Each picture is tiny: **8 × 8 pixels**. Each **pixel** (one dot of the picture) is a number from 0 (white) to 16 (black).

```
from sklearn.datasets import load_digits

digits = load_digits()

print("images:", digits.images.shape)    # 1797 pictures of 8 × 8 pixels
print("labels:", digits.target.shape)    # the right answer for each picture
print("first 10 labels:", digits.target[:10])
```

```
images: (1797, 8, 8)
labels: (1797,)
first 10 labels: [0 1 2 3 4 5 6 7 8 9]
```

Here is the very first picture, as Python sees it. Can you spot the shape of a 0 in the big numbers?

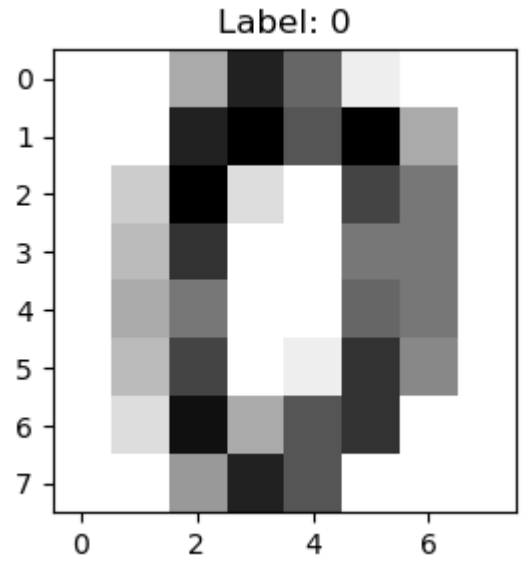
```
digits.images[0]
```

```
array([[ 0.,  0.,  5., 13.,  9.,  1.,  0.,  0.],
       [ 0.,  0., 13., 15., 10., 15.,  5.,  0.],
       [ 0.,  3., 15.,  2.,  0., 11.,  8.,  0.],
       [ 0.,  4., 12.,  0.,  0.,  8.,  8.,  0.],
       [ 0.,  5.,  8.,  0.,  0.,  9.,  8.,  0.],
       [ 0.,  4., 11.,  0.,  1., 12.,  7.,  0.],
       [ 0.,  2., 14.,  5., 10., 12.,  0.,  0.],
       [ 0.,  0.,  6., 13., 10.,  0.,  0.,  0.]])
```

`plt.imshow` draws a 2D array as a picture: each number becomes a square of colour. `cmap="gray_r"` picks a reversed grey scale, so 0 is white and 16 is black, like ink on paper:

```
import matplotlib.pyplot as plt

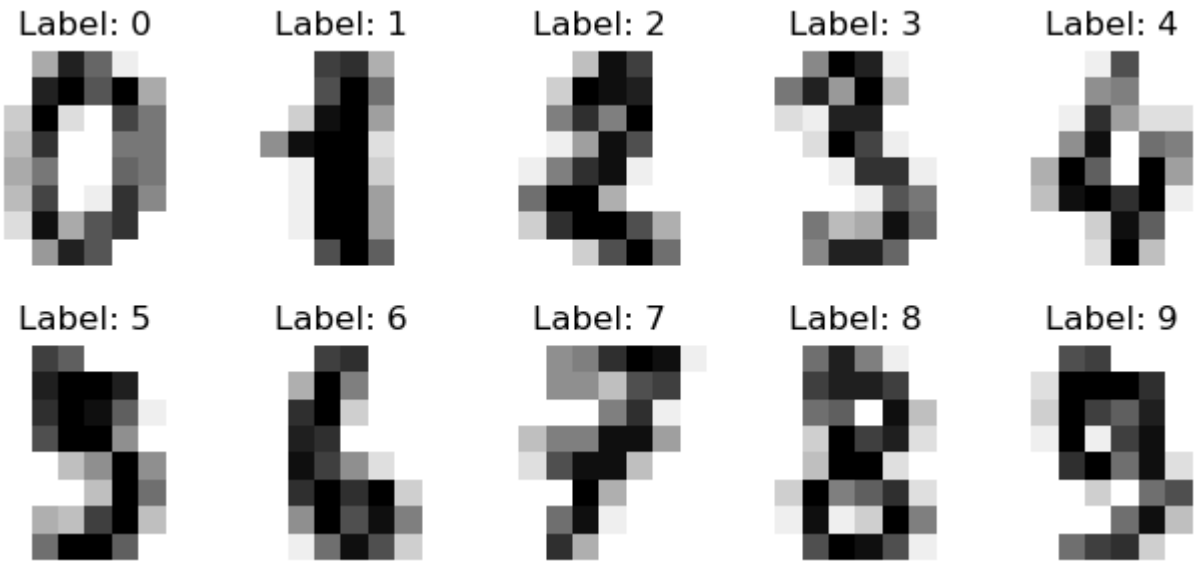
plt.figure(figsize=(3, 3))
plt.imshow(digits.images[0], cmap="gray_r")
plt.title(f"Label: {digits.target[0]}")
plt.show()
```



Let's look at the first 10 digits together. `plt.subplots(2, 5)` makes a grid of 2 × 5 small charts; `axes.ravel()` turns the grid into one flat row so we can loop over it:

```
fig, axes = plt.subplots(2, 5, figsize=(8, 3.5))

for ax, image, label in zip(axes.ravel(), digits.images, digits.target):
    ax.imshow(image, cmap="gray_r")
    ax.set_title(f"Label: {label}")
    ax.axis("off")            # hide the axis numbers
plt.show()
```



📏 From a grid to 64 features

The models you know expect each example as **one row of features**, like a row in a table. So we **flatten** each 8 × 8 grid into one row of 64 numbers: row 1 of the picture, then row 2, and so on. `.reshape(64)` does that, and the dataset already has the flat version ready in `digits.data`:

```
flat = digits.images[0].reshape(64)
print(flat)
print("same as digits.data[0]?", (flat == digits.data[0]).all())
print("digits.data:", digits.data.shape)    # 1797 rows × 64 features
```

```
[ 0.  0.  5. 13.  9.  1.  0.  0.  0.  0. 13. 15. 10. 15.  5.  0.  0.  3.
 15.  2.  0. 11.  8.  0.  0.  4. 12.  0.  0.  8.  8.  0.  0.  5.  8.  0.
  0.  9.  8.  0.  0.  4. 11.  0.  1. 12.  7.  0.  0.  2. 14.  5. 10. 12.
  0.  0.  0.  0.  6. 13. 10.  0.  0.  0.]
same as digits.data[0]? True
digits.data: (1797, 64)
```

One more step: **scaling**. Neural networks learn best when the inputs are small, roughly between 0 and 1 (remember yesterday: big inputs make big gradient steps, and the learning can jump around). Dividing by 16 turns 0–16 into 0–1. Then we split into training and test sets as on Day 16, keeping the same mix of digits in both (`stratify=y`):

```
from sklearn.model_selection import train_test_split

X = digits.data / 16      # pixels now go from 0 to 1
y = digits.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y)

print("train:", X_train.shape, " test:", X_test.shape)
print("smallest and biggest pixel:", X.min(), X.max())
```

```
train: (1347, 64)  test: (450, 64)
smallest and biggest pixel: 0.0 1.0
```

🧠 A neural network that reads digits

scikit-learn's `MLPClassifier` is a ready-made neural network. MLP stands for **multi-layer perceptron**: layers of neurons, just like yesterday. Ours will have:

- **64 inputs**: one per pixel.
- **One hidden layer of 64 neurons**: `hidden_layer_sizes=(64,)`. (Two hidden layers would be `(64, 32)`.)
- **10 outputs**: one per digit. The biggest one wins.

A few things are different from our hand-made network, but the idea is the same:

Word	What it means
ReLU	The hidden neurons' activation: `z` if it's positive, otherwise 0. Simpler than sigmoid and learns faster in big networks.
softmax	Turns the 10 output scores into 10 probabilities that add up to 1.
adam	The optimiser: gradient descent that adjusts its own step size for each weight.
mini-batch	Each step uses a small batch of 200 pictures instead of all of them, so steps are quick.
`max_iter`	The number of epochs (passes through the training data).

After 200 epochs the network may still be improving a tiny bit, and scikit-learn would print a `ConvergenceWarning` ("not fully finished"). That's fine here, so we hide that one warning. Training takes a few seconds:

```
import warnings
from sklearn.exceptions import ConvergenceWarning
from sklearn.neural_network import MLPClassifier

warnings.filterwarnings("ignore", category=ConvergenceWarning)

mlp = MLPClassifier(hidden_layer_sizes=(64,), max_iter=200, random_state=42)
mlp.fit(X_train, y_train)

print(f"train accuracy: {mlp.score(X_train, y_train):.3f}")
print(f"test accuracy:  {mlp.score(X_test, y_test):.3f}")
```

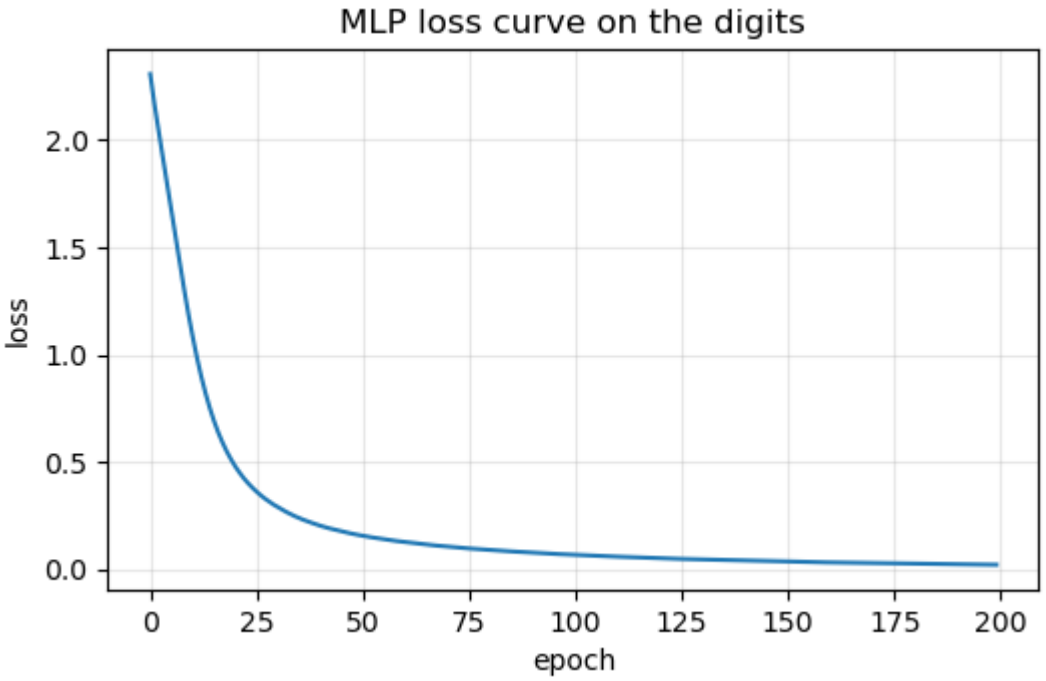
```
train accuracy: 0.999
test accuracy:  0.982
```

About 98% of pictures it has **never seen** are read correctly. Let's peek inside: how many weights did it learn, and what did its loss curve look like?

```
print("input → hidden weights:", mlp.coefs_[0].shape)
print("hidden → output weights:", mlp.coefs_[1].shape)
# all weights, plus one bias per neuron (64 hidden + 10 output)
total = mlp.coefs_[0].size + mlp.coefs_[1].size + 64 + 10
print("numbers learned in total:", total)

plt.figure(figsize=(6, 3.5))
plt.plot(mlp.loss_curve_)
plt.xlabel("epoch")
plt.ylabel("loss")
plt.title("MLP loss curve on the digits")
plt.grid(alpha=0.3)
plt.show()
```

```
input → hidden weights: (64, 64)
hidden → output weights: (64, 10)
numbers learned in total: 4810
```



4810 numbers, all found by gradient descent. (Big modern networks have billions.) The loss shown is not MSE but **log-loss**, the usual loss for classification: it punishes the network for being confident and wrong.

🔮 Making predictions

```
pred = mlp.predict(X_test)

print("predicted:", pred[:15])
print("true:     ", y_test[:15])
print("correct:  ", (pred == y_test).sum(), "of", len(y_test))
```

```
predicted: [1 0 9 1 5 6 7 9 1 5 5 9 2 2 7]
true:      [1 0 9 1 5 6 7 9 1 5 5 9 2 2 7]
correct:   442 of 450
```

To predict one single picture, a common slip is to pass just that one row:

💡 This example has a mistake on purpose. Run it and read the error.

```
mlp.predict(X_test[0])
```

⚠️ ValueError

The kind of value is right but the value itself isn't, like int("hello").

```
Traceback (most recent call last):
  line 1, in <module>
    mlp.predict(X_test[0])
    ~~~~~~~~~~~^^^^^^^^^^^
ValueError: Expected 2D array, got 1D array instead:
array=[0.     0.     0.5    1.     0.9375 0.5    0.     0.     0.     0.0625
 1.     1.     1.     0.125  0.     0.     0.     0.125  1.     1.
 0.625  0.     0.     0.     0.     0.125  1.     1.     0.75   0.
 0.     0.     0.     0.375  1.     1.     0.8125 0.     0.     0.
 0.     0.0625 1.     1.     0.6875 0.     0.     0.     0.     0.
 1.     1.     0.625  0.     0.     0.     0.     0.     0.4375 0.9375
 0.9375 0.     0.     0.    ].
Reshape your data either using array.reshape(-1, 1) if your data has a single
feature or array.reshape(1, -1) if it contains a single sample.
```

`X_test[0]` is a flat list of 64 numbers (1D), but the model always wants a **table** (2D): rows of examples. The error message even says how to fix it. A table with 1 row and 64 columns is `X_test[0].reshape(1, 64)`:

```
one_picture = X_test[0].reshape(1, 64)
print(one_picture.shape)
print("prediction:", mlp.predict(one_picture), " true:", y_test[0])
```

```
(1, 64)
prediction: [1]  true: 1
```

🔍 Which digits get mixed up?

The **confusion matrix** from Day 20 works with 10 classes too: one row per true digit, one column per predicted digit. The diagonal holds the correct answers; every other number is a mistake.

```
import pandas as pd
from sklearn.metrics import confusion_matrix

cm = confusion_matrix(y_test, pred)

cm_table = pd.DataFrame(cm)
cm_table.index.name = "true"
cm_table.columns.name = "predicted"
cm_table
```

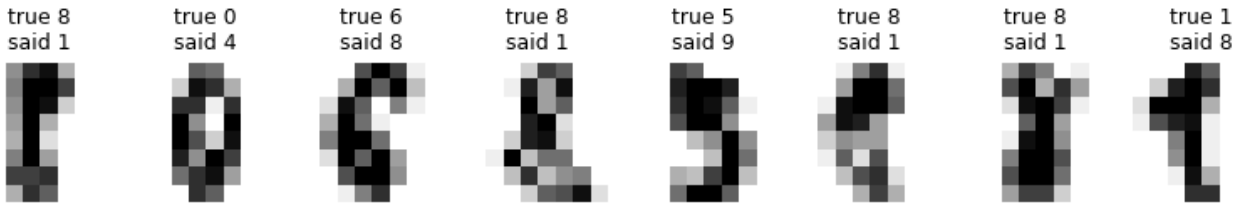
predicted	0	1	2	3	4	5	6	7	8	9
true										
0	44	0	0	0	1	0	0	0	0	0
1	0	45	0	0	0	0	0	0	1	0
2	0	0	44	0	0	0	0	0	0	0
3	0	0	0	46	0	0	0	0	0	0
4	0	0	0	0	45	0	0	0	0	0
5	0	0	0	0	0	45	0	0	0	1
6	0	0	0	0	0	0	44	0	1	0
7	0	0	0	0	0	0	0	45	0	0
8	0	4	0	0	0	0	0	0	39	0
9	0	0	0	0	0	0	0	0	0	45

Numbers are good, pictures are better. Let's draw the test pictures it got wrong. A boolean mask (Day 9) picks them out:

```
wrong = pred != y_test
wrong_images = X_test[wrong]
wrong_true = y_test[wrong]
wrong_pred = pred[wrong]
print("mistakes:", wrong.sum())

fig, axes = plt.subplots(1, 8, figsize=(10, 2))
for i in range(8):
    axes[i].axis("off")
    if i < len(wrong_images):
        axes[i].imshow(wrong_images[i].reshape(8, 8), cmap="gray_r")
        title = f"true {wrong_true[i]}\nsaid {wrong_pred[i]}"
        axes[i].set_title(title, fontsize=9)
plt.show()
```

```
mistakes: 8
```



Many of these are honestly hard to read, even for a person. With only 8 × 8 pixels, a sloppy 8 can look like a 1.

📊 How sure is the network?

`predict_proba` gives the 10 probabilities (from softmax) for each picture. The biggest one is the network's **confidence** in its answer. `.max(axis=1)` takes the biggest number in each row. Is the network less sure when it's wrong?

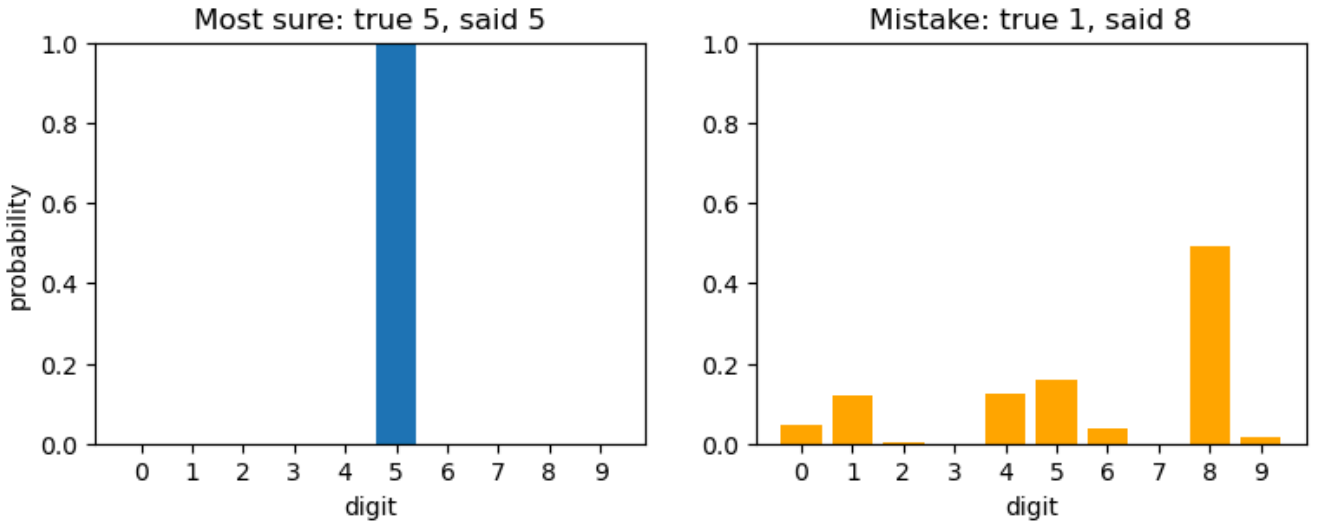
```
probs = mlp.predict_proba(X_test)     # 10 probabilities per picture
confidence = probs.max(axis=1)        # how sure it was about its answer

print(f"average confidence when right: {confidence[pred == y_test].mean():.2f}")
print(f"average confidence when wrong: {confidence[wrong].mean():.2f}")

sure = confidence.argmax()            # its most confident picture
unsure = confidence[wrong].argmin()   # its least confident mistake

fig, (left, right) = plt.subplots(1, 2, figsize=(9, 3))
left.bar(range(10), probs[sure])
left.set_title(f"Most sure: true {y_test[sure]}, said {pred[sure]}")
right.bar(range(10), probs[wrong][unsure], color="orange")
right.set_title(f"Mistake: true {wrong_true[unsure]}, said {wrong_pred[unsure]}")
for ax in (left, right):
    ax.set_xticks(range(10))
    ax.set_xlabel("digit")
    ax.set_ylim(0, 1)
left.set_ylabel("probability")
plt.show()
```

```
average confidence when right: 0.97
average confidence when wrong: 0.75
```



On average the network is clearly less sure when it's wrong, and for its shakiest mistake the probability is spread over several digits. But not always: a few mistakes are made with high confidence. In real systems, pictures with low confidence are often sent to a human to check.

⚖️ Is the network worth it?

Always compare with a simpler model (Day 20's checklist). Logistic regression (Day 18) is like a network with **no hidden layer**:

```
from sklearn.linear_model import LogisticRegression

logreg = LogisticRegression(max_iter=1000)
logreg.fit(X_train, y_train)

print(f"logistic regression: {logreg.score(X_test, y_test):.3f}")
print(f"neural network:      {mlp.score(X_test, y_test):.3f}")
```

```
logistic regression: 0.962
neural network:      0.982
```

The hidden layer helps, but the simple model is already good on these tiny, clean pictures. On big photos, networks with many layers win by a huge margin. Also notice the network's **train** accuracy (almost 100%) is higher than its test accuracy: it remembers its training pictures a bit too well. That gap is tomorrow's topic.

⚠️ Common mistakes

- Forgetting to scale the pixels (`/ 16`): the network learns slower and worse.
- Passing one picture as a 1D row → `ValueError: Expected 2D array`. Use `.reshape(1, 64)`.
- Passing `digits.images` (8 × 8 grids) to the model instead of `digits.data` (flat rows of 64).
- Judging the model on its training pictures. Only the test accuracy tells you how it does on new handwriting.
- Forgetting `random_state=42`: the network starts from random weights, so results change a little each run.

✅ Recap

- A greyscale picture is a grid of numbers; `plt.imshow(image, cmap="gray_r")` shows it.
- Flatten 8 × 8 into 64 features (`.reshape(64)` or `digits.data`) and scale to 0–1.
- `MLPClassifier(hidden_layer_sizes=(64,), max_iter=200, random_state=42)`, then `.fit`, `.score`, `.predict`.
- The confusion matrix shows which digits get mixed up; looking at the mistakes shows why.
- `predict_proba` tells you how confident the network is.
- Always compare with a simpler model.

🏋️ Exercises

Write your answer in the space under each exercise. Try it on the computer too, by opening this day's `index.html`. Solutions are in the answer key at the end.

1. How big is the dataset?

Using `.shape` of `digits.data`, store the number of pictures in `n_images` and the number of features per picture in `n_pixels`. Also store the label of the **last** picture in `last_label`.

```
from sklearn.datasets import load_digits

digits = load_digits()

# Store n_images, n_pixels and last_label
```

Hints:

1. `digits.data.shape` is a pair: (rows, columns). Get the parts with `[0]` and `[1]`.
2. The labels are in `digits.target`; the last item of an array is at index `-1`.
3. `n_images = digits.data.shape[0]`, `n_pixels = digits.data.shape[1]`, `last_label = digits.target[-1]`

2. Show a digit

Store picture number `index` (an 8 × 8 grid from `digits.images`) in `image` and its label in `label`. Then draw it with `plt.imshow` (in grey) with the label as the title.

```
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits

digits = load_digits()
index = 42

# Store image and label, then show the picture
```

Hints:

1. `image = digits.images[index]` and `label = digits.target[index]`
2. `plt.imshow(image, cmap="gray_r")` draws it.
3. `plt.title(f"Label: {label}")` adds the title.

3. Train your own network

The data is prepared for you. Create an `MLPClassifier` called `model` with **32** hidden neurons, `max_iter=100` and `random_state=42`, train it on the training set, and store its **test** accuracy in `accuracy`. It should reach at least 0.9.

```
import warnings
from sklearn.exceptions import ConvergenceWarning
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier

warnings.filterwarnings("ignore", category=ConvergenceWarning)

digits = load_digits()
X = digits.data / 16
y = digits.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y)

# Create model, fit it, store accuracy
```

Hints:

1. `model = MLPClassifier(hidden_layer_sizes=(32,), max_iter=100, random_state=42)` (note the comma in `(32,)`).
2. Train with `model.fit(X_train, y_train)`.
3. `accuracy = model.score(X_test, y_test)`

4. Count the mistakes

A small network is trained for you and its predictions are in `pred`. Store how many test pictures it got **wrong** in `n_mistakes`, and the percentage that is wrong in `error_rate` (for example 3.1 for 3.1%).

```
import warnings
from sklearn.exceptions import ConvergenceWarning
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier

warnings.filterwarnings("ignore", category=ConvergenceWarning)

digits = load_digits()
X_train, X_test, y_train, y_test = train_test_split(
    digits.data / 16, digits.target,
    test_size=0.25, random_state=42, stratify=digits.target)
model = MLPClassifier(hidden_layer_sizes=(32,), max_iter=100, random_state=42)
model.fit(X_train, y_train)
pred = model.predict(X_test)

# Store n_mistakes and error_rate
```

Hints:

1. `pred != y_test` is a boolean array: True where the network was wrong.
2. `.sum()` counts the Trues: `n_mistakes = (pred != y_test).sum()`
3. `error_rate = n_mistakes / len(y_test) * 100`

5. Challenge: the most confused pair

Using the confusion matrix `cm` (rows = true digit, columns = predicted digit), find the most common mistake: the biggest number that is **not** on the diagonal. Store its true digit in `true_digit` and the predicted digit in `predicted_digit`.

```
import warnings
from sklearn.exceptions import ConvergenceWarning
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import confusion_matrix

warnings.filterwarnings("ignore", category=ConvergenceWarning)

digits = load_digits()
X_train, X_test, y_train, y_test = train_test_split(
    digits.data / 16, digits.target,
    test_size=0.25, random_state=42, stratify=digits.target)
model = MLPClassifier(hidden_layer_sizes=(32,), max_iter=100, random_state=42)
model.fit(X_train, y_train)
cm = confusion_matrix(y_test, model.predict(X_test))
print(cm)

# Find true_digit and predicted_digit
```

Hints:

1. Loop over every row `t` and every column `p` (two `for` loops over `range(10)`), and skip the diagonal with `if t != p`.
2. Keep track of the biggest count so far in a variable (start it at -1), and remember its `t` and `p` when you find a bigger one.
3. Inside the loops: `if t != p and cm[t, p] > biggest:` then `biggest = cm[t, p]`, `true_digit = t`, `predicted_digit = p`.

➡️ Tomorrow

Overfitting, regularisation and real-world frameworks: why a model can be perfect on its training data and still fail on new data, and the tricks that stop it (regularisation, early stopping, more data). Then a first look at PyTorch and Keras, the tools professionals use, and how their pieces match what you built by hand.

🔑 Answer key

Try the exercises yourself first! There is often more than one correct answer.

1. How big is the dataset?

```
from sklearn.datasets import load_digits

digits = load_digits()

n_images = digits.data.shape[0]
n_pixels = digits.data.shape[1]
last_label = digits.target[-1]
print(n_images, n_pixels, last_label)
```

2. Show a digit

```
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits

digits = load_digits()
index = 42

image = digits.images[index]
label = digits.target[index]
plt.figure(figsize=(3, 3))
plt.imshow(image, cmap="gray_r")
plt.title(f"Label: {label}")
plt.show()
```

3. Train your own network

```
import warnings
from sklearn.exceptions import ConvergenceWarning
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier

warnings.filterwarnings("ignore", category=ConvergenceWarning)

digits = load_digits()
X = digits.data / 16
y = digits.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y)

model = MLPClassifier(hidden_layer_sizes=(32,), max_iter=100, random_state=42)
model.fit(X_train, y_train)
accuracy = model.score(X_test, y_test)
print(f"test accuracy: {accuracy:.3f}")
```

4. Count the mistakes

```
import warnings
from sklearn.exceptions import ConvergenceWarning
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier

warnings.filterwarnings("ignore", category=ConvergenceWarning)

digits = load_digits()
X_train, X_test, y_train, y_test = train_test_split(
    digits.data / 16, digits.target,
    test_size=0.25, random_state=42, stratify=digits.target)
model = MLPClassifier(hidden_layer_sizes=(32,), max_iter=100, random_state=42)
model.fit(X_train, y_train)
pred = model.predict(X_test)

n_mistakes = (pred != y_test).sum()
error_rate = n_mistakes / len(y_test) * 100
print(f"{n_mistakes} mistakes, {error_rate:.1f}% wrong")
```

5. Challenge: the most confused pair

```
import warnings
from sklearn.exceptions import ConvergenceWarning
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import confusion_matrix

warnings.filterwarnings("ignore", category=ConvergenceWarning)

digits = load_digits()
X_train, X_test, y_train, y_test = train_test_split(
    digits.data / 16, digits.target,
    test_size=0.25, random_state=42, stratify=digits.target)
model = MLPClassifier(hidden_layer_sizes=(32,), max_iter=100, random_state=42)
model.fit(X_train, y_train)
cm = confusion_matrix(y_test, model.predict(X_test))
print(cm)

biggest = -1
for t in range(10):
    for p in range(10):
        if t != p and cm[t, p] > biggest:
            biggest = cm[t, p]
            true_digit = t
            predicted_digit = p
print(f"Most common mistake: {true_digit} read as {predicted_digit}, {biggest}
times")
```